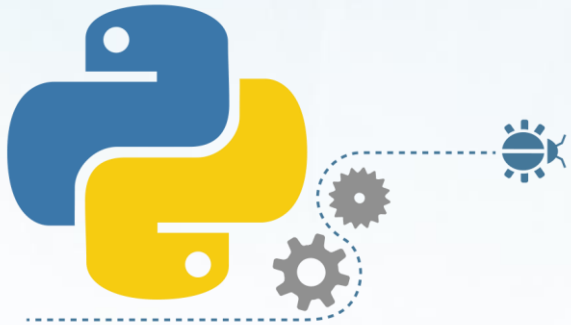
TRƯỜNG ĐẠI HỌC
SƯ PHẠM KỸ THUẬT TP. HỒ CHÍ MINH
HCMC University of Technology and Education

KHOA CÔNG NGHỆ THÔNG TIN
BỘ MÔN HỆ THỐNG THÔNG TIN

NHẬP MÔN LẬP TRÌNH PYTHON (IPPA233277)

BIỂU DIỄN DỮ LIỆU MATPLOTLIB



1. Nắm được các khái niệm và thành phần trong matplotlib
2. Nắm được các loại biểu đồ hỗ trợ trong matplotlib
3. Vận dụng được matplotlib trong vẽ các loại biểu đồ



1. Giới thiệu matplotlib
2. Các loại biểu đồ
3. Làm việc với văn bản
4. Vẽ nhiều đối tượng trong cùng một biểu đồ
5. Vẽ trên nhiều phân vùng



- Dùng để trực quan hóa dữ liệu hoặc đơn giản là vẽ các đồ thị hàm số 2D từ dữ liệu mảng cho mục đích minh họa
- Thư viện matplotlib cho phép vẽ các hình ảnh bằng các chấm các điểm ảnh trên cửa sổ vẽ rồi nối các điểm lại với nhau để tạo thành hình ảnh cuối cùng
- Cài đặt matplotlib : **conda install matplotlib / pip install matplotlib**
- Khai báo thư viện : **from matplotlib import pyplot as plt / import matplotlib.pyplot as plt**

- Các kiểu plot:
  - ✓ Bar: bar plot
  - ✓ Barh: Lập biểu đồ thanh ngang.
  - ✓ Boxplot: box và whisker plot
  - ✓ Hist: Vẽ biểu đồ
  - ✓ Hist2d: Vẽ biểu đồ 2D
  - ✓ Pie: Vẽ biểu đồ hình tròn
  - ✓ Quiver: Vẽ trường mũi tên 2-D
  - ✓ Polar: vẽ polar plot
  - ✓ Scatter: Lập biểu đồ phân tán của x vs y
  - ✓ Stackplot: Vẽ một biểu đồ khu vực xếp chồng lên nhau
  - ✓ Stem: Tạo stem plot
  - ✓ Step: Lập sơ đồ từng bước
  - ✓ Plot: Vẽ các đường và / hoặc điểm đánh dấu cho Axes
- Hàm image
  - ✓ imread: đọc một hình ảnh từ một tập tin vào một mảng
  - ✓ imsave: lưu một mảng dưới dạng tập tin hình ảnh
  - ✓ imshow: hiển thị một hình ảnh lên các trục



- Trục Axis:
  - ✓ Axes: Thêm axes vào hình
  - ✓ Text: Thêm text vào axes
  - ✓ Title: Đặt tiêu đề của các trục
  - ✓ Xlabel: Đặt trục x của trục hiện tại
  - ✓ Xlim: Lấy hoặc đặt giới hạn x của các trục
  - ✓ Xscale
  - ✓ Xticks: Nhận hoặc đặt giới hạn x của các vị trí và nhãn đánh dấu
  - ✓ Ylabel: Đặt axis y của trục
  - ✓ Ylim: Nhận hoặc đặt giới hạn y của các trục
  - ✓ Yscale: Đặt tỷ lệ của trục y.
  - ✓ Yticks: Nhận hoặc đặt giới hạn y của các vị trí và nhãn đánh dấu
- Hàm Figure:
  - ✓ Figtext: thêm text vào figure
  - ✓ Figure: Tạo figture mới
  - ✓ Show: Hiển thị figture
  - ✓ Savefig: Lưu trữ figure hiện tại
  - ✓ Close: Đóng cửa sổ figture



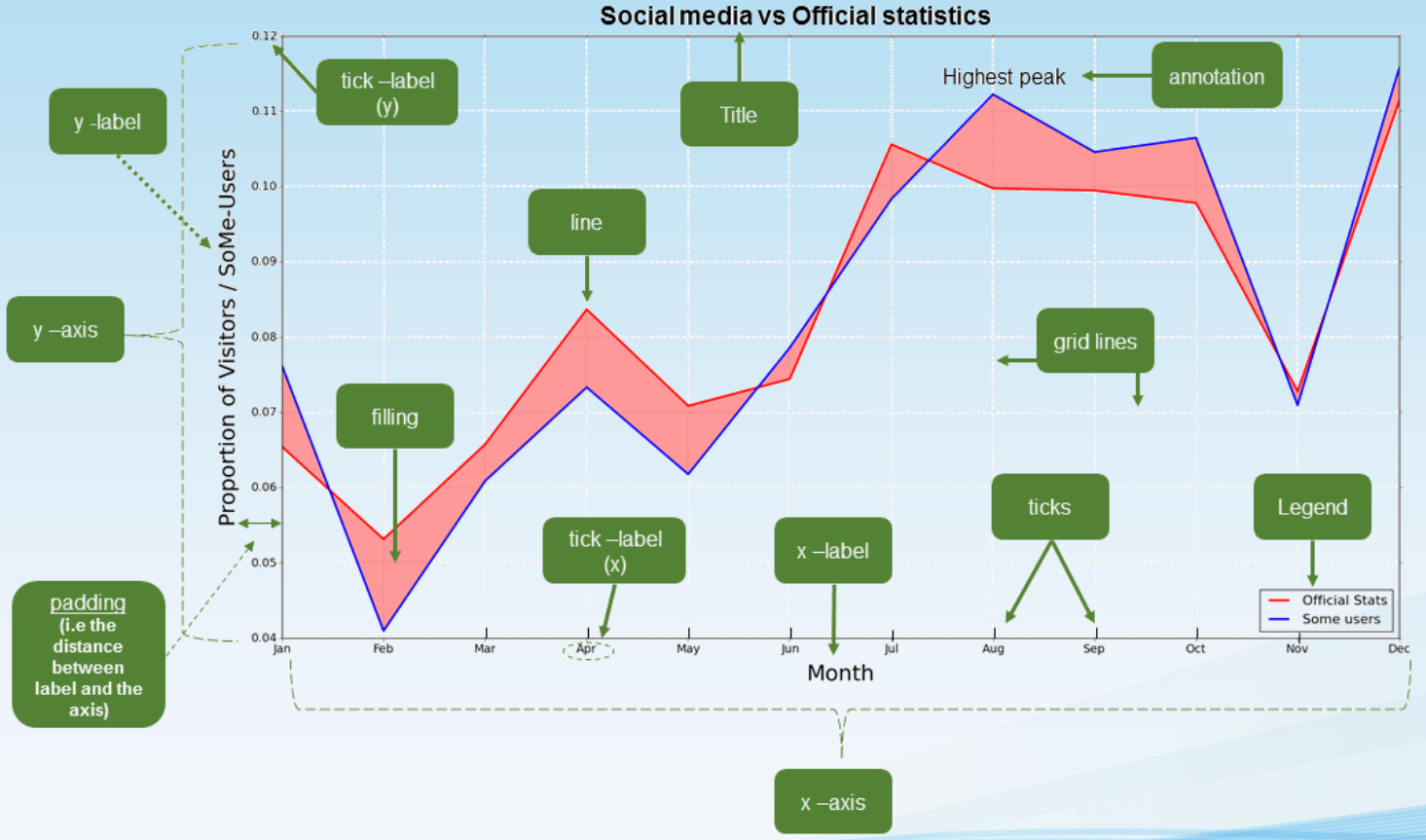
- **Vẽ hình**
  - ✓ plt.plot(x, y) # x, y là các mảng/ma trận

- **Giao diện hướng đối tượng**
  - ✓ fig = plt.figure([left, bottom, width, height]) # tạo khung hình
  - ✓ ax = fig.add_axes() # thêm trục
  - ✓ ax.set_title() | ax.set_xlabel() | ax.set_ylabel() → ax.plot()
  - ✓ ax.legend(handles, labels, loc) # bổ sung chú giải cho hình

Char	Color	Char	Description	Char	Description
'b'	Blue	'.'	Point marker	'-'	Solid line
'g'	Green	'o'	Circle marker	'--'	Dashed line
'r'	Red	'x'	X marker	'-.'	Dash-dot line
'c'	Cyan	'D'	Diamond marker	':'	Dotted line
'm'	Magenta	'H'	Hexagon marker	'+'	Plus marker
'y'	Yellow	's'	Square marker		
'k'	Black	'w'	White		

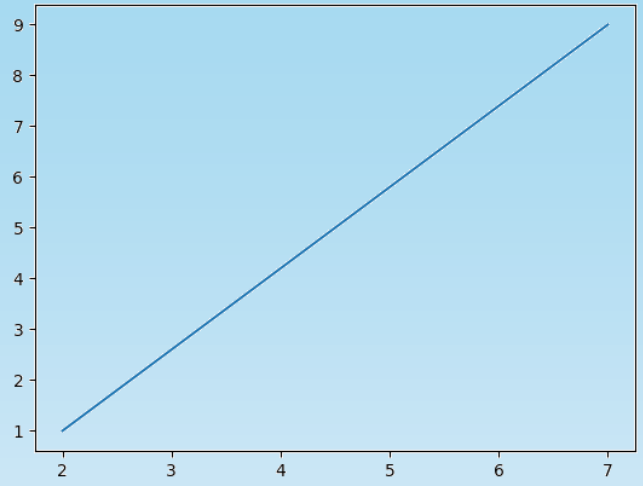


- Biểu đồ là một biểu diễn đồ họa để trực quan hóa dữ liệu, đại diện cho dữ liệu dạng bảng, các chức năng và cung cấp các thông tin khác nhau liên quan đến dữ liệu
- Trực quan hóa dữ liệu thông qua biểu đồ giúp khám phá mẫu, xu hướng, mối quan hệ và cấu trúc trong dữ liệu
- Các thành phần biểu đồ (hình bên)

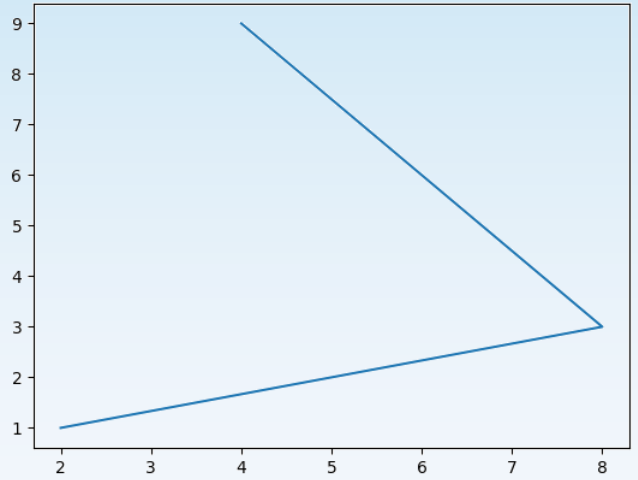


- **Vẽ đoạn thẳng**

```
x = np.array([2, 7])
y = np.array([1, 9])
plt.plot(x,y)
plt.show()
```

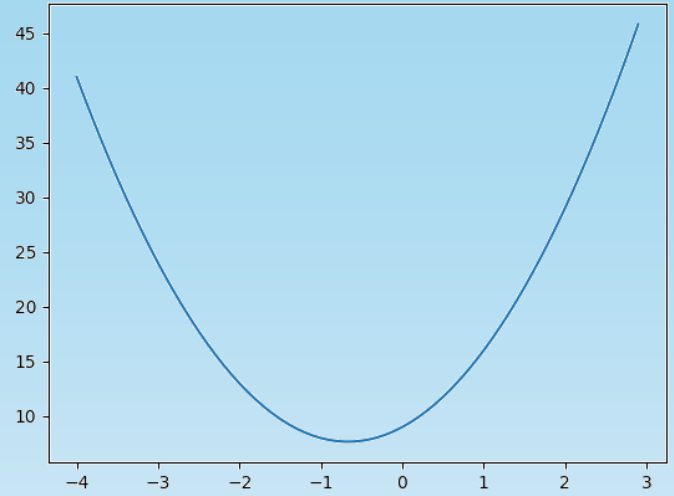



```
x = np.array([2, 8, 4])
y = np.array([1, 3, 9])
plt.plot(x,y)
plt.show()
```



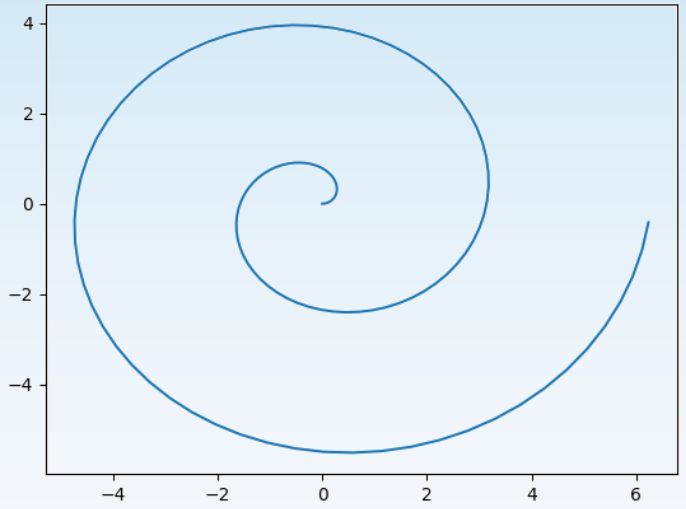
- **Vẽ đồ thị $y = 3x^2 + 4x + 9$**

```
x = np.arange(-4, 3, 0.1)
y = 3*x**2 + 4*x + 9
plt.plot(x,y)
plt.show()
```



- **Vẽ hình xoắn ốc $y = 0.5\theta$**

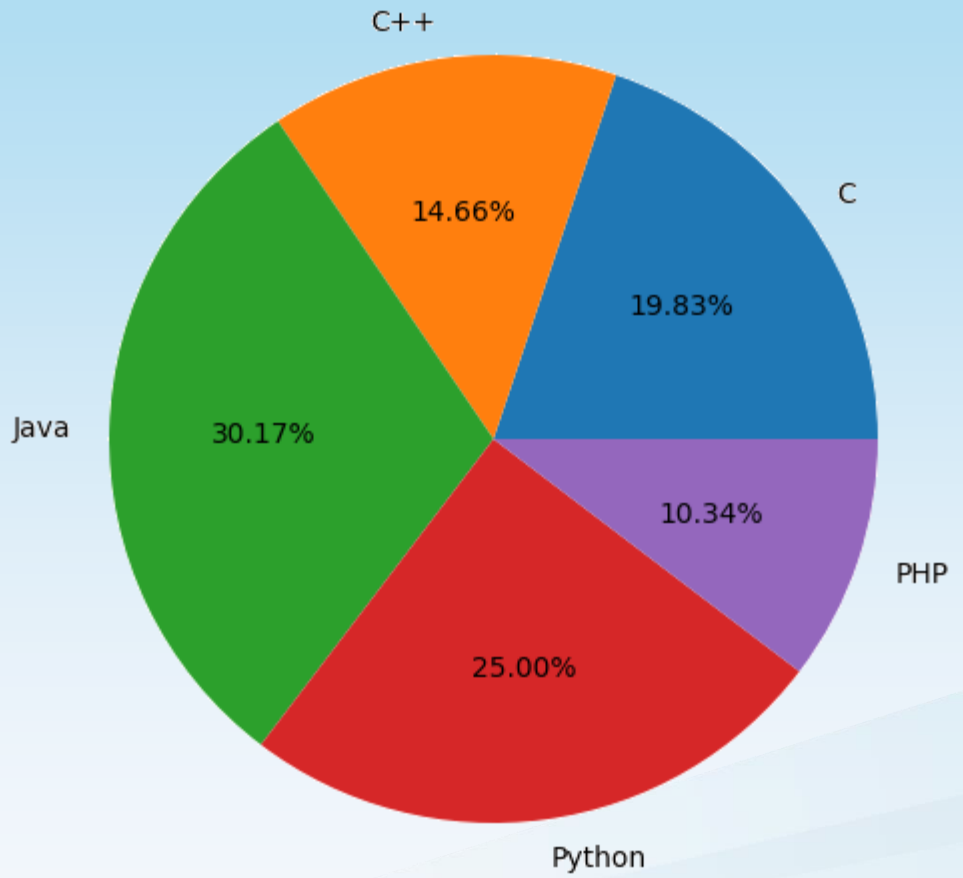
```
theta = np.arange(0, 4*np.pi ,0.1)
r = 0.5*theta
x = r*np.cos(theta)
y = r*np.sin(theta)
plt.plot(x,y)
plt.show()
```



- Biểu đồ thể hiện sự so sánh các phần với toàn bộ dùng để biểu diễn cơ cấu theo phần trăm. Toàn bộ hình tròn đại diện cho tổng số và mỗi phần nhỏ đại diện cho một mục cụ thể trong tổng thể.
- Thường được sử dụng cơ cấu, tỉ trọng, quy mô, tỷ lệ

Ví dụ:

```
fig = plt.figure()
ax = fig.add_axes([0,0,1,1])
ax.axis('equal')
langs = ['C', 'C++', 'Java', 'Python', 'PHP']
students = [23,17,35,29,12]
ax.pie(students, labels = langs,autopct='%1.2f%%')
plt.show()
```



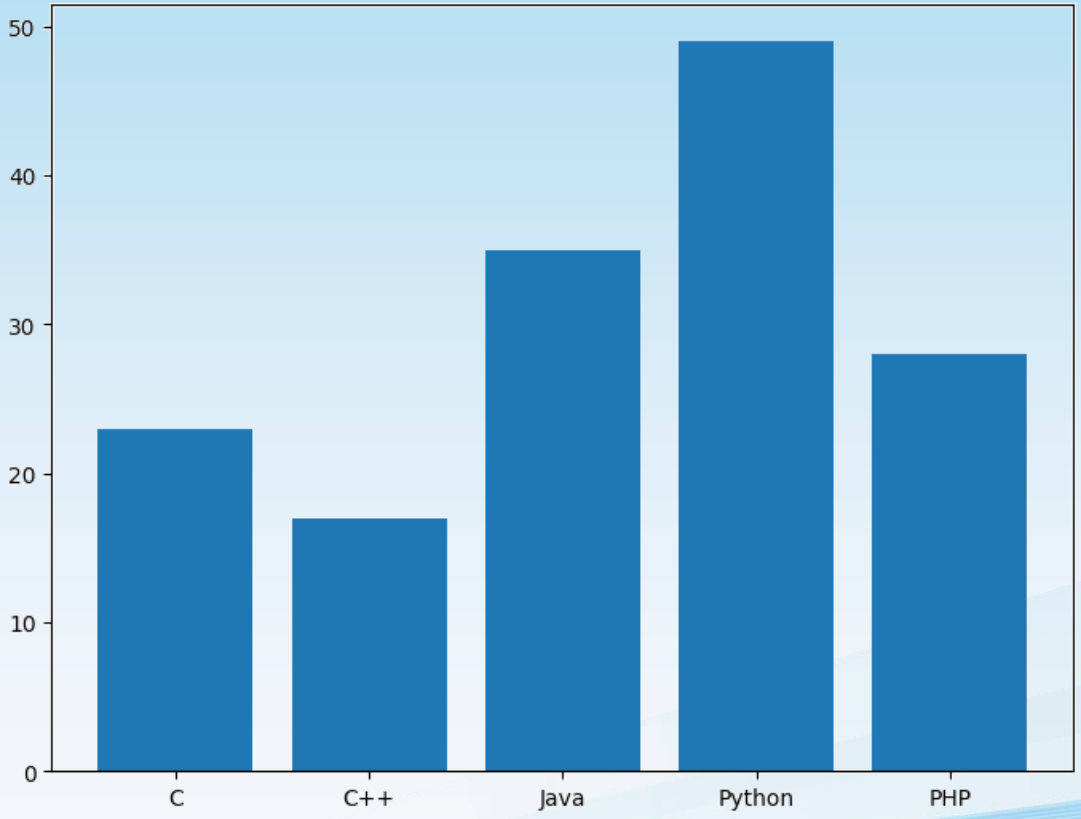
- Một số dạng biểu đồ khác của biểu đồ tròn
  - ✓ Donut chart
  - ✓ Stacked donut chart
  - ✓ Semi-Circle pie chart
  - ✓ Irregular pie chart
- Một số lưu ý khi dùng pie chart
  - ✓ Đảm bảo tổng các thành phần là 100%
  - ✓ Chỉ dùng pie chart khi số lượng thành phần ít hơn 6
  - ✓ Không dùng pie chart nếu tỷ lệ giữa các thành phần gần tương đương nhau
  - ✓ Nên sắp xếp các giá trị để dễ dàng so sánh hiểu về dữ liệu
  - ✓ Tránh sử dụng dạng 3D hoặc định dạng biểu đồ nghiêng



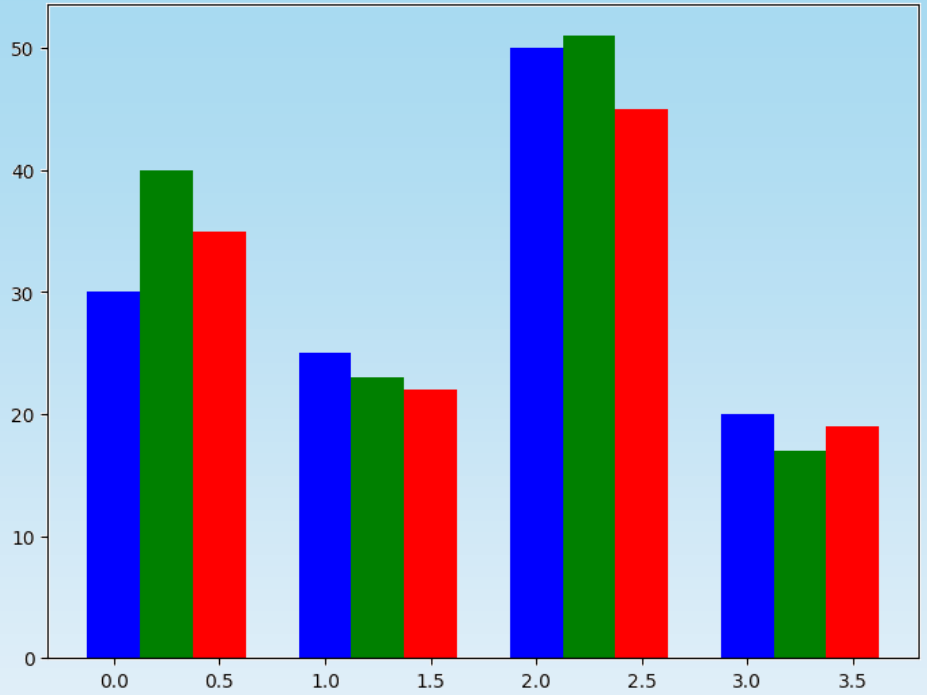
- Biểu thị giá trị của các biến dưới dạng các thanh được sắp xếp song song để so sánh giá trji giữa các biến, đồng thời hiểu rõ hơn về sự phân bố của một tập dữ liệu
- Trục đứng thường biểu thị giá trị của biến, trong khi trục ngang địa diện cho tên hoặc loại của biến
- Vẽ biểu đồ hình cột sử dụng cú pháp

```
ax.bar(x, height, width, bottom, align)
```

```
fig = plt.figure()
ax = fig.add_axes([0,0,1,1])
langs = ['C', 'C++', 'Java', 'Python', 'PHP']
students = [23,17,35,29,48]
ax.bar(langs,students)
plt.show()
```



```
data = [[30, 25, 50, 20],
[40, 23, 51, 17],
[35, 22, 45, 19]]
X = np.arange(4)
fig = plt.figure()
ax = fig.add_axes([0,0,1,1])
ax.bar(X + 0.00, data[0], color = 'b', width = 0.25)
ax.bar(X + 0.25, data[1], color = 'g', width = 0.25)
ax.bar(X + 0.50, data[2], color = 'r', width = 0.25)
plt.show()
```

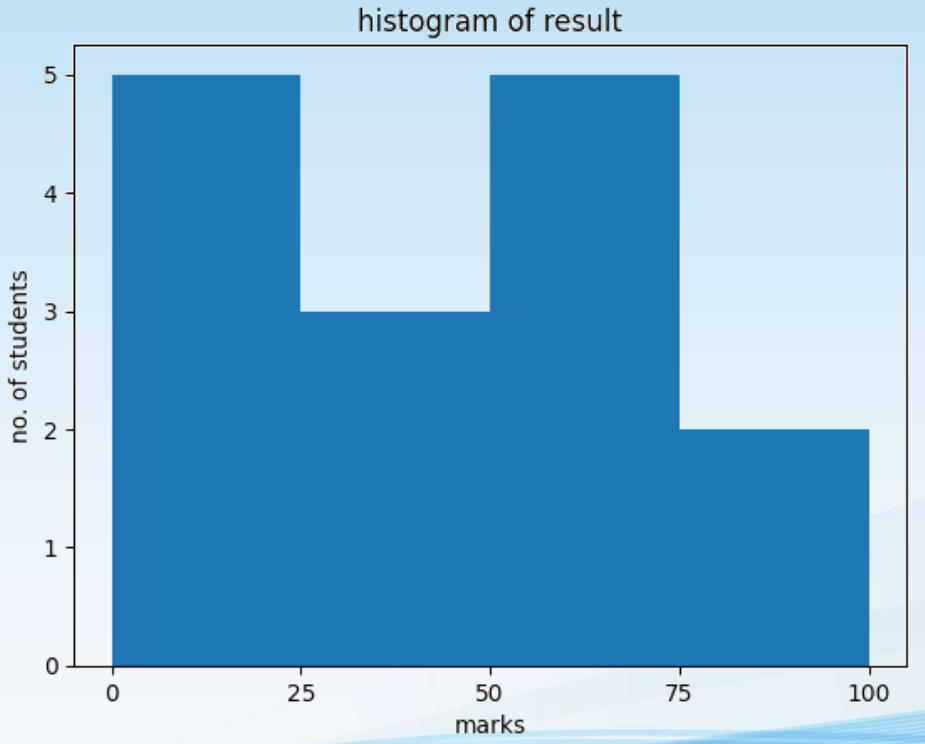


- Một số dạng biểu đồ khác của biểu đồ cột
  - ✓ Biểu đồ thanh ngang (horizontal bar chart)
  - ✓ Biểu đồ thanh xếp chồng (stacked bar chart)
  - ✓ Biểu đồ thanh xếp chồng 100%

- Là biểu đồ phân bố dữ liệu số, ước lượng phân phối của một biến liên tục và là dạng biểu đồ thanh

```
ax.hist(a, bins)
```

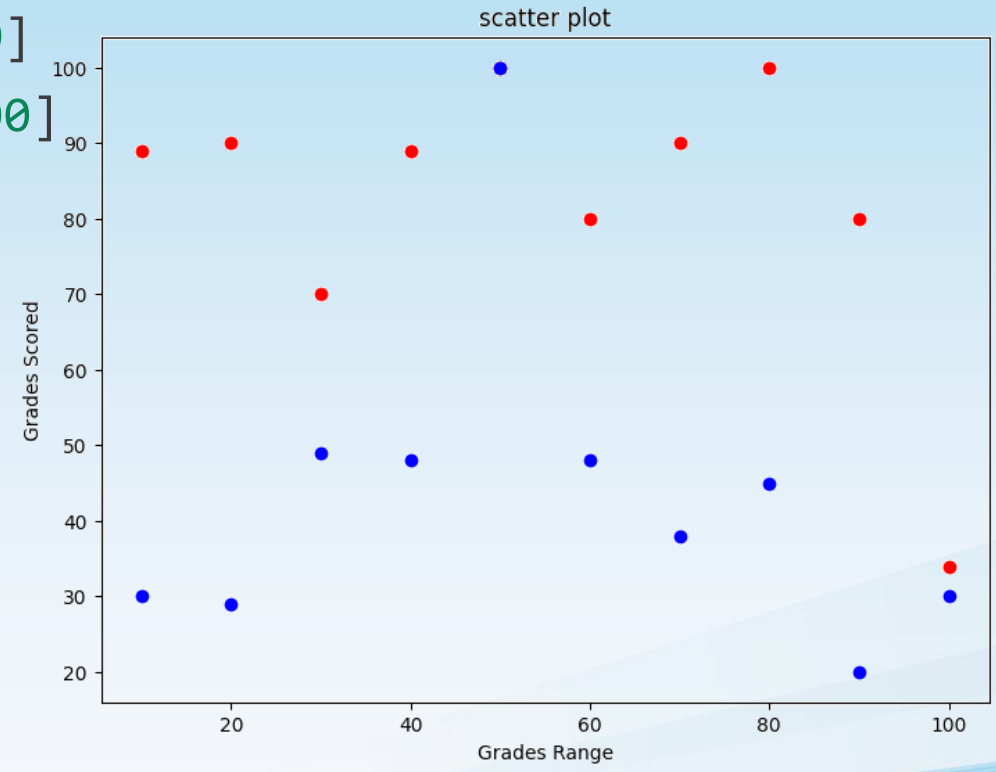
```
fig,ax = plt.subplots(1,1)
a = np.array([22,87,5,43,56,73,55,54,11,20,51,5,79,31,27])
ax.hist(a, bins = [0,25,50,75,100])
ax.set_title("histogram of result")
ax.set_xticks([0,25,50,75,100])
ax.set_xlabel('marks')
ax.set_ylabel('no. of students')
plt.show()
```



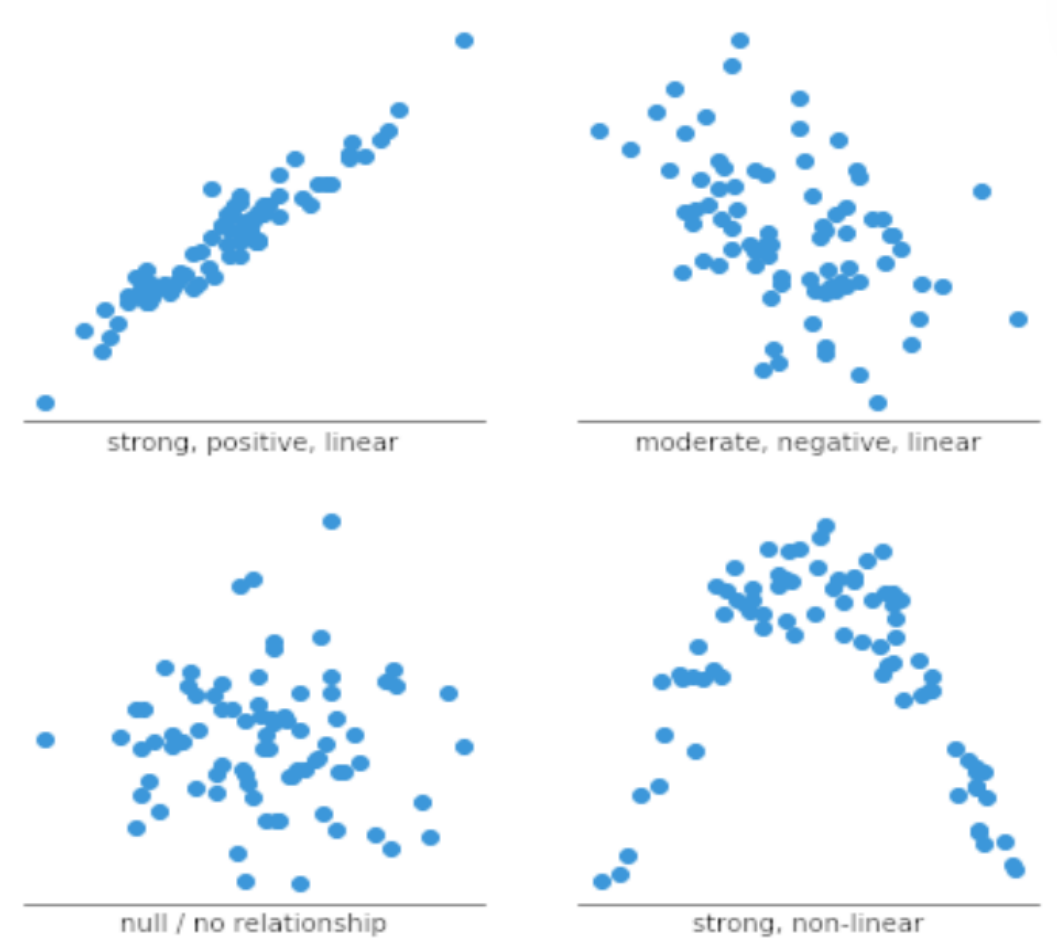
- Gồm các điểm dữ liệu được đặt trên một thang chia, vị trí mỗi điểm trên trục ngang và trục dọc biểu thị các giá trị của một điểm, được sử dụng để quan sát mối quan hệ giữa các biến
- Để vẽ các điểm dữ liệu trên trục hoành và trục tung để thể hiện mức độ ảnh hưởng của một biến này tới biến khác.

```
ax.scatter(x, y)
```

```
girls_grades = [89, 90, 70, 89, 100, 80, 90, 100, 80, 34]
boys_grades = [30, 29, 49, 48, 100, 48, 38, 45, 20, 30]
grades_range = [10, 20, 30, 40, 50, 60, 70, 80, 90, 100]
fig=plt.figure()
ax=fig.add_axes([0,0,1,1])
ax.scatter(grades_range, girls_grades, color='r')
ax.scatter(grades_range, boys_grades, color='b')
ax.set_xlabel('Grades Range')
ax.set_ylabel('Grades Scored')
ax.set_title('scatter plot')
plt.show()
```



- Các thức sử dụng



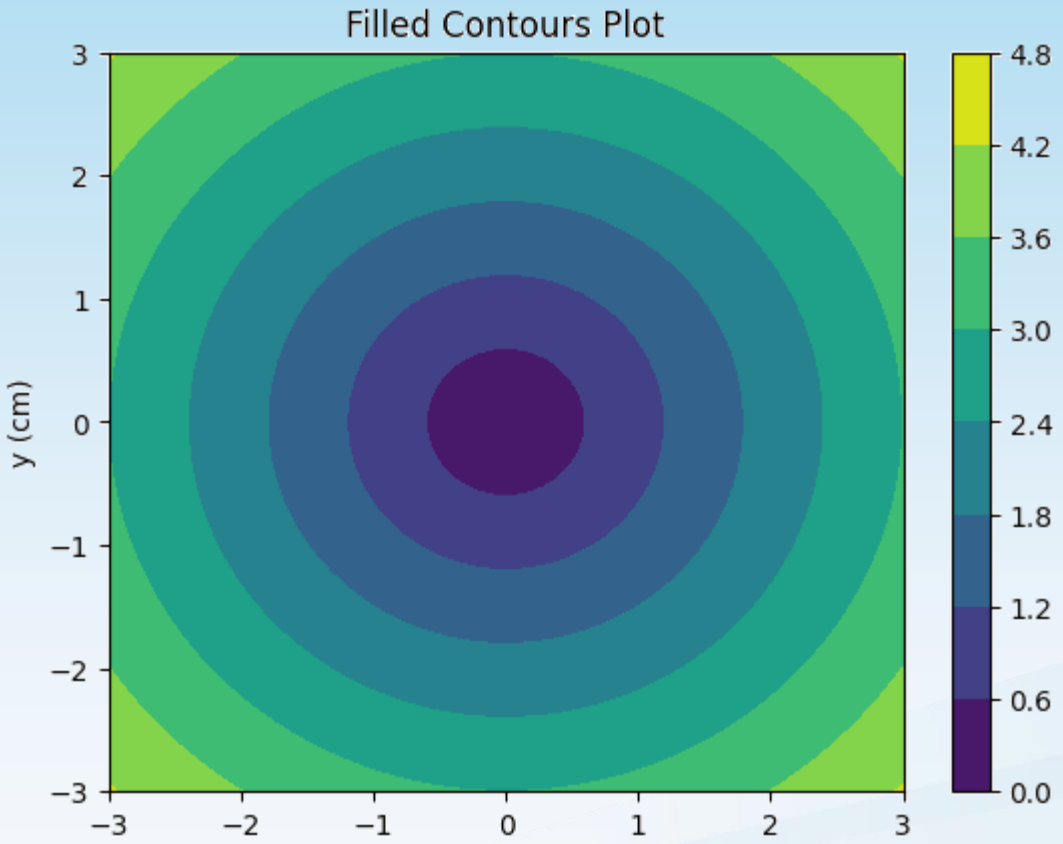


- Còn gọi là đồ thị mức là một cách thể hiện một bề mặt 3 chiều trên một mặt phẳng 2 chiều

```
ax.contour[f]([X, Y,] Z, [levels], **kwargs)
```

Trong đó: đường viền Z thay đổi như thế nào dưới dạng hàm của hai đầu vào X và Y, sao cho Z = f (X, Y)

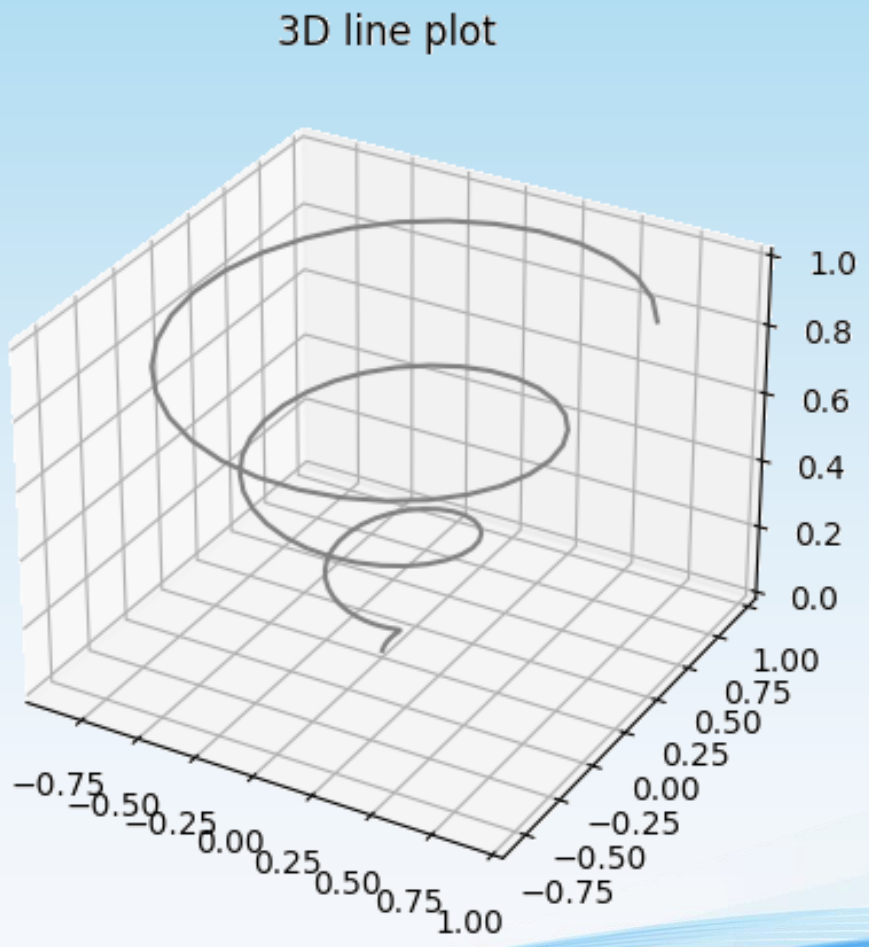
```
xlist = np.linspace(-3.0, 3.0, 100)
ylist = np.linspace(-3.0, 3.0, 100)
X, Y = np.meshgrid(xlist, ylist)
Z = np.sqrt(X**2 + Y**2)
fig,ax=plt.subplots(1,1)
cp = ax.contourf(X, Y, Z)
fig.colorbar(cp) # Add a colorbar to a plot
ax.set_title('Filled Contours Plot')
#ax.set_xlabel('x (cm)')
ax.set_ylabel('y (cm)')
plt.show()
```



- Plot 3 chiều được hỗ trợ trong bộ công cụ mplot3d

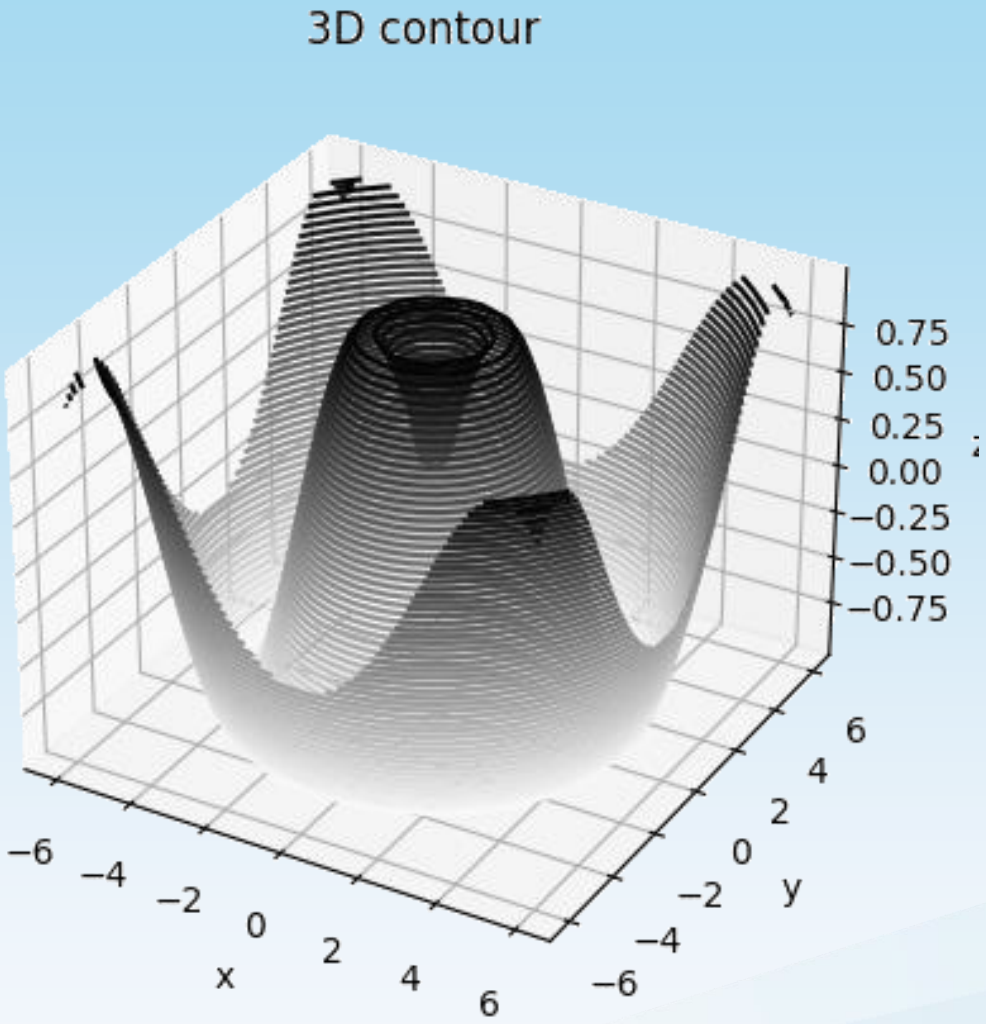
```
from mpl_toolkits import mplot3d
```

```
from mpl_toolkits import mplot3d
import numpy as np
import matplotlib.pyplot as plt
fig = plt.figure()
ax = plt.axes(projection='3d')
z = np.linspace(0, 1, 100)
x = z * np.sin(20 * z)
y = z * np.cos(20 * z)
ax.plot3D(x, y, z, 'gray')
ax.set_title('3D line plot')
plt.show()
```

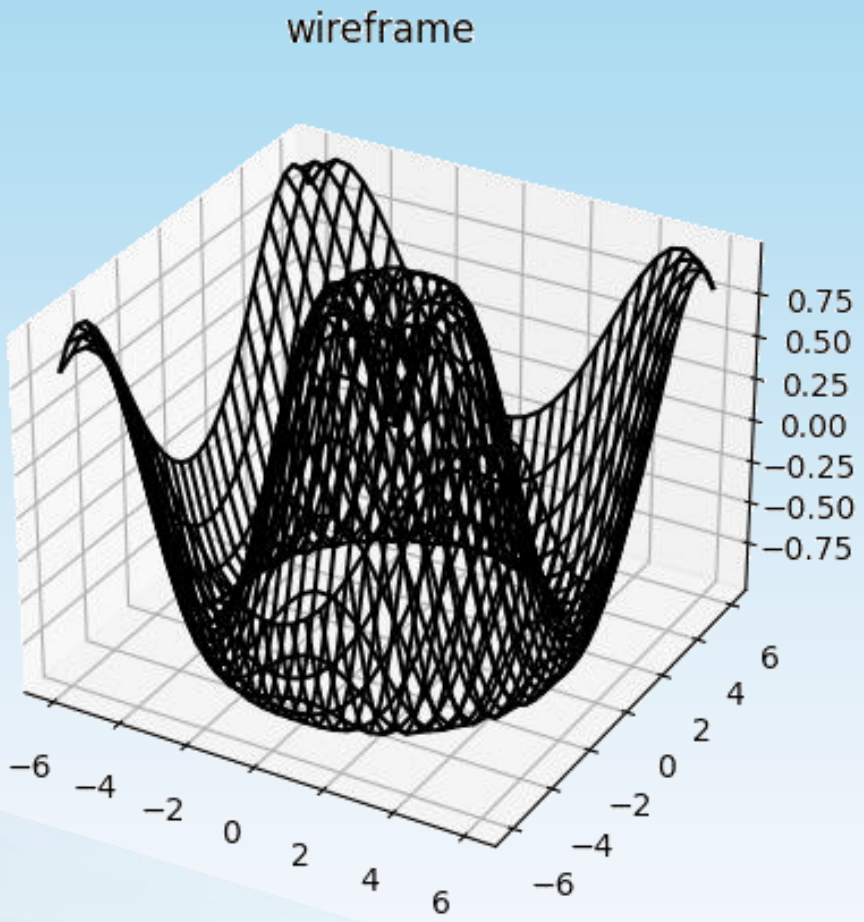


- Hàm ax.contour3D () tạo biểu đồ đường bao ba chiều

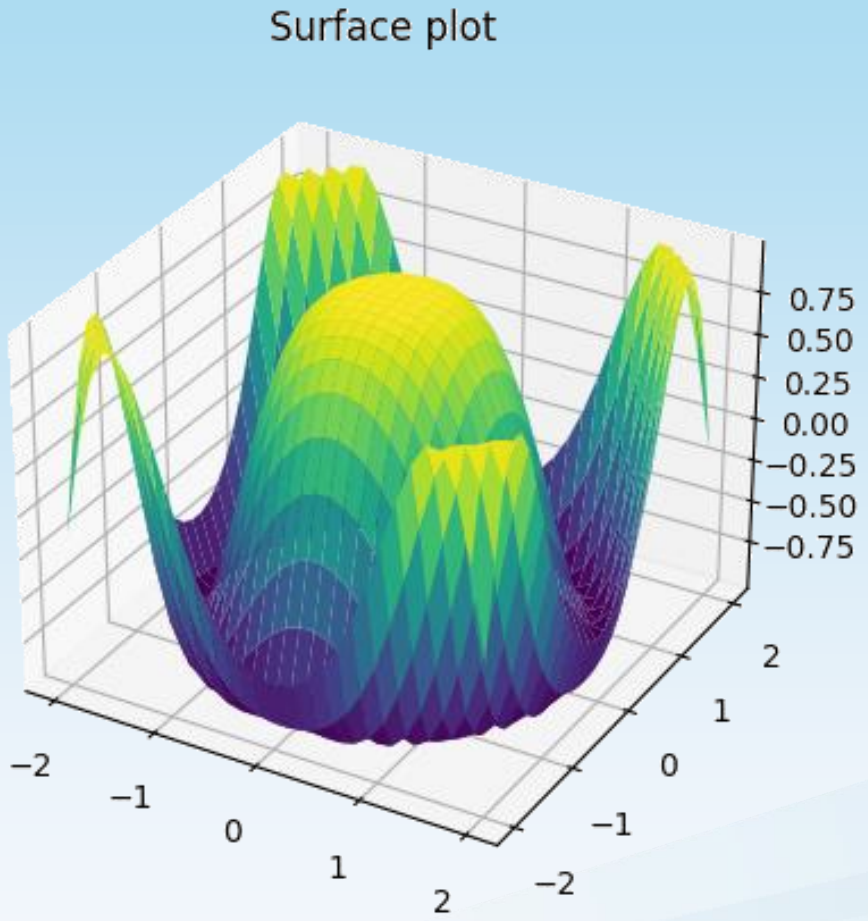
```
def f(x, y):
    return np.sin(np.sqrt(x ** 2 + y ** 2))
x = np.linspace(-6, 6, 30)
y = np.linspace(-6, 6, 30)
X, Y = np.meshgrid(x, y)
Z = f(X, Y)
fig = plt.figure()
ax = plt.axes(projection='3d')
ax.contour3D(X, Y, Z, 50, cmap='binary')
ax.set_xlabel('x')
ax.set_ylabel('y')
ax.set_zlabel('z')
ax.set_title('3D contour')
plt.show()
```



- `ax.plot_wireframe(X, Y, Z, color='black')`



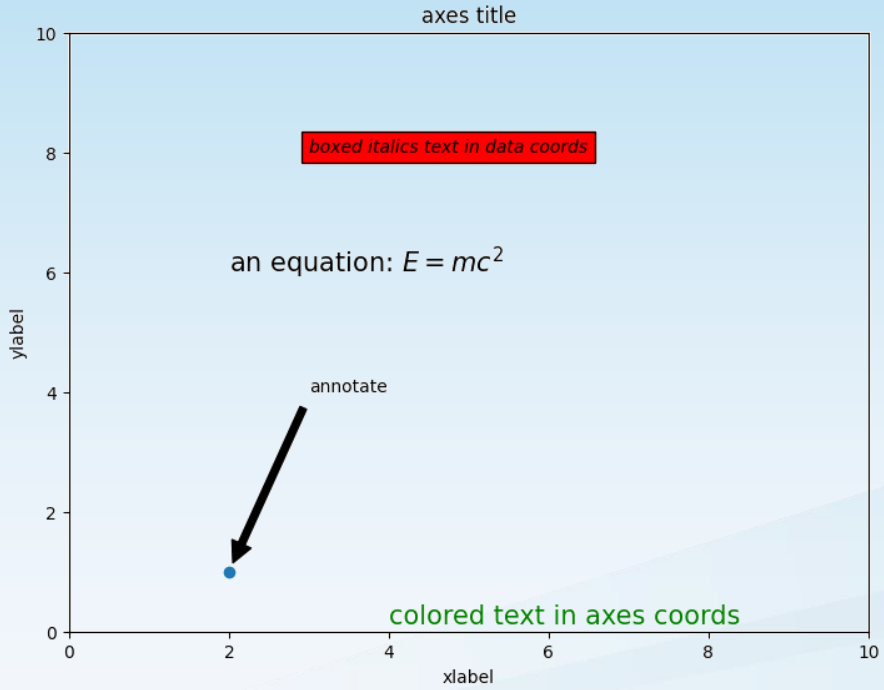
- `ax.plot_surface(x, y, z,cmap='viridis', edgecolor='none')`



- Matplotlib hỗ trợ văn bản, biểu diễn các biểu thức toán học

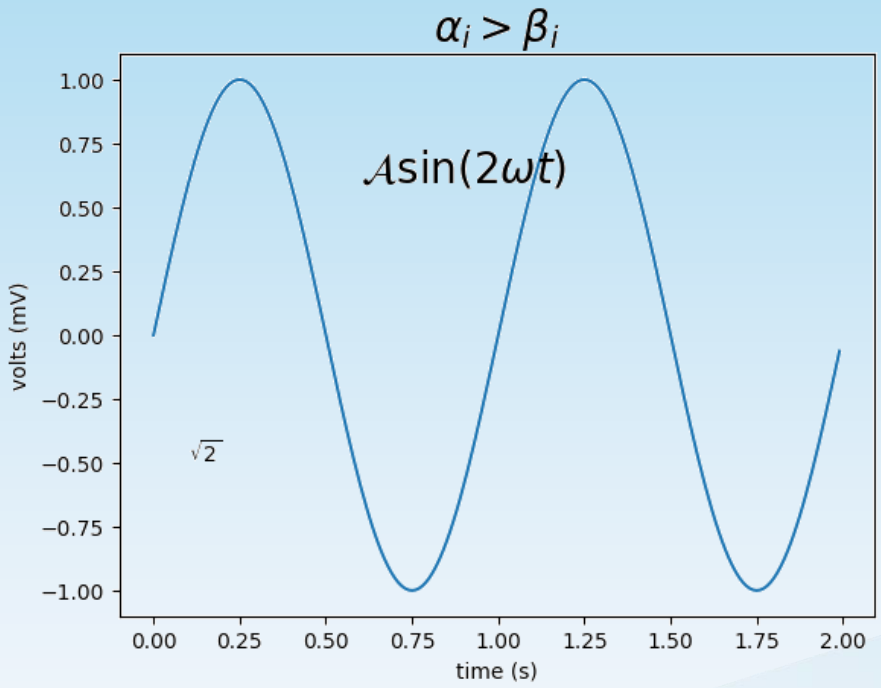
	Mô tả		Mô tả
figtext	Thêm văn bản vào một vị trí tùy ý của Figure.	xlabel	Thêm nhãn vào trục x của Axes.
suptitle	Thêm tiêu đề vào Figure	ylabel	Thêm nhãn vào trục y của Axes.
text	Thêm văn bản tại một vị trí tùy ý của Axes.	title	Thêm title vào Axes
annotate	Thêm chú thích, với một mũi tên tùy chọn trên Axes.		

```
fig = plt.figure()
ax = fig.add_axes([0,0,1,1])
ax.set_title('axes title’); ax.set_xlabel('xlabel’); ax.set_ylabel('ylabel')
ax.text(3, 8, 'boxed italics text in data coords’,
style='italic', bbox = {'facecolor': 'red'})
ax.text(2, 6, r'an equation: $E = mc^2$', fontsize = 15)
ax.text(4, 0.05, 'colored text in axes coords',
verticalalignment = 'bottom', color = 'green', fontsize = 15)
ax.plot([2], [1], 'o')
ax.annotate('annotate', xy = (2, 1), xytext = (3, 4),
arrowprops = dict(facecolor = 'black', shrink = 0.05))
ax.axis([0, 10, 0, 10])
plt.show()
```



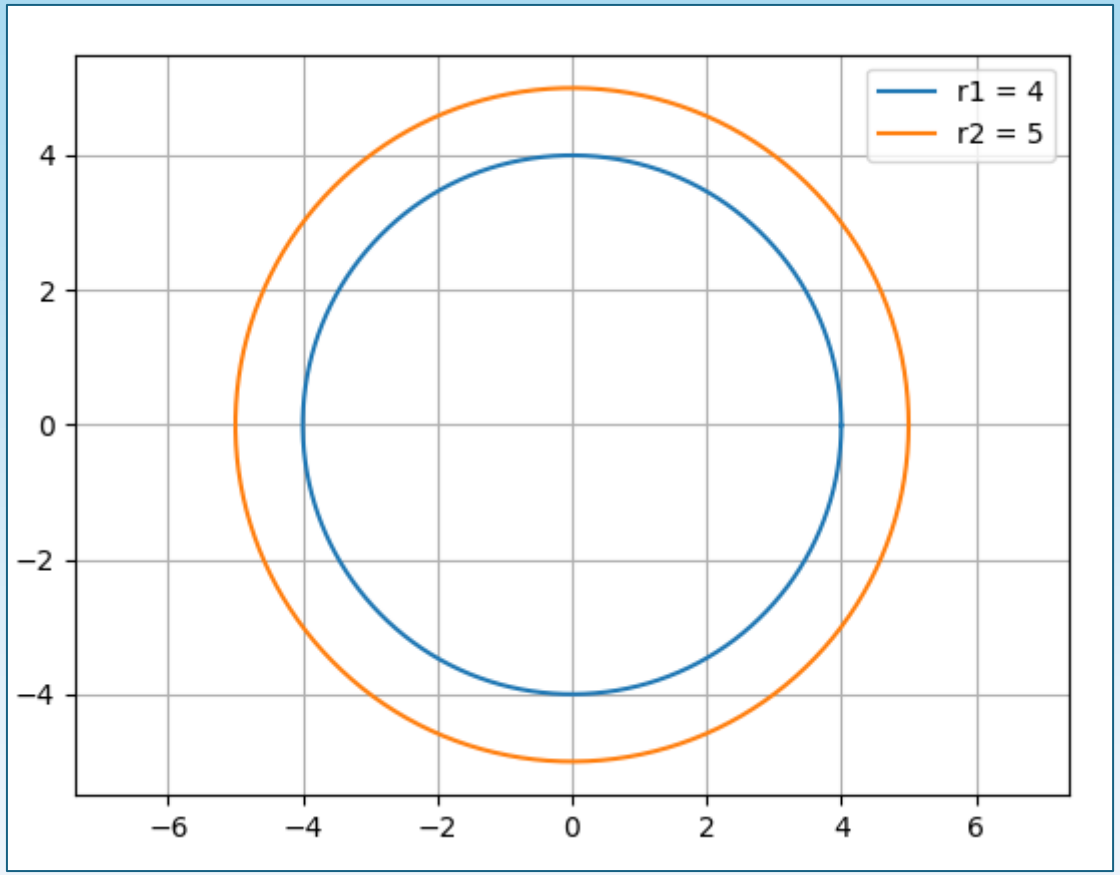
- Sử dụng TeXmarkup trong bất kỳ chuỗi văn bản Matplotlib nào bằng cách đặt bên trong một cặp dấu đô la ($)

```
r'$\alpha_i> \beta_i$'
t = np.arange(0.0, 2.0, 0.01)
s = np.sin(2*np.pi*t)
plt.plot(t,s)
plt.title(r'$\alpha_i> \beta_i$', fontsize=20)
plt.text(0.6, 0.6,
r'$\mathcal{A}\mathrm{sin}(2 \omega t)$', fontsize = 20)
plt.text(0.1, -0.5, r'$\sqrt{2}$', fontsize=10)
plt.xlabel('time (s)')
plt.ylabel('volts (mV)')
plt.show()
```



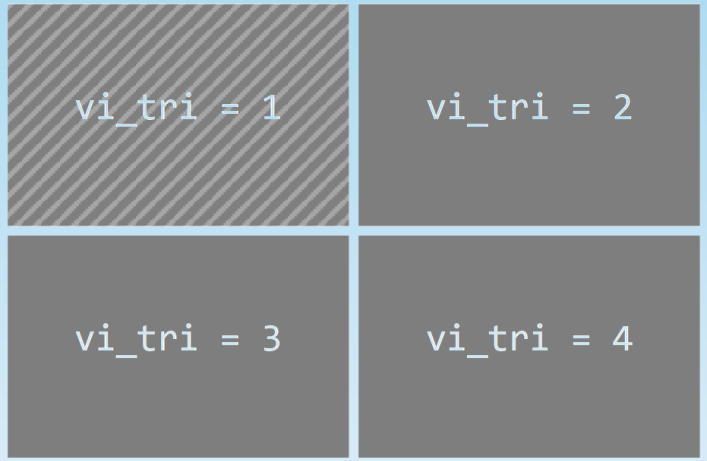
- **Vẽ hai đường tròn**

```
theta = np.arange(0, 2*np.pi ,0.01)
r = 4
x = r*np.cos(theta)
y = r*np.sin(theta)
plt.plot(x,y)
r = 5
x = r*np.cos(theta)
y = r*np.sin(theta)
plt.plot(x,y)
plt.legend(['r1 = 4','r2 = 5'])
plt.axis('equal')
plt.grid(True)
plt.show()
```



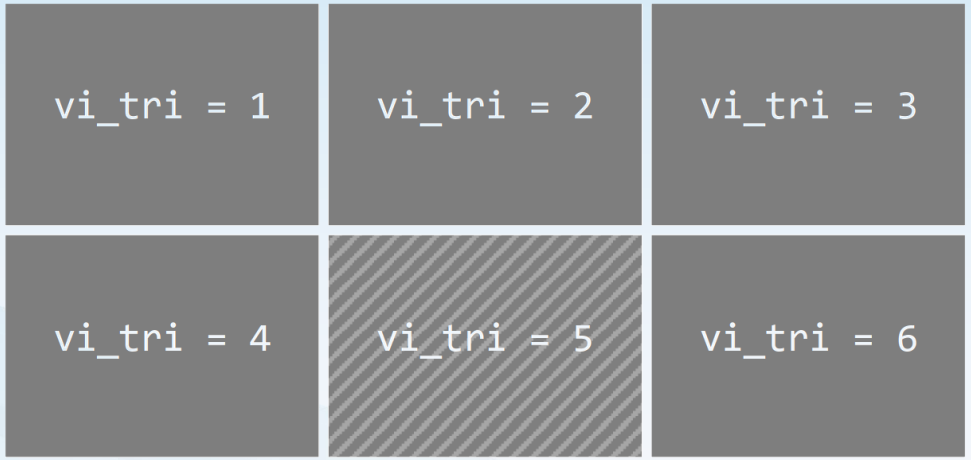
- Để tách một cửa sổ vẽ (figure) thành nhiều phân vùng, sử dụng hàm subplotlib với cú pháp:

```
subplot(nrows, ncols, index)
add_subplot(nrows, ncols, index)
```

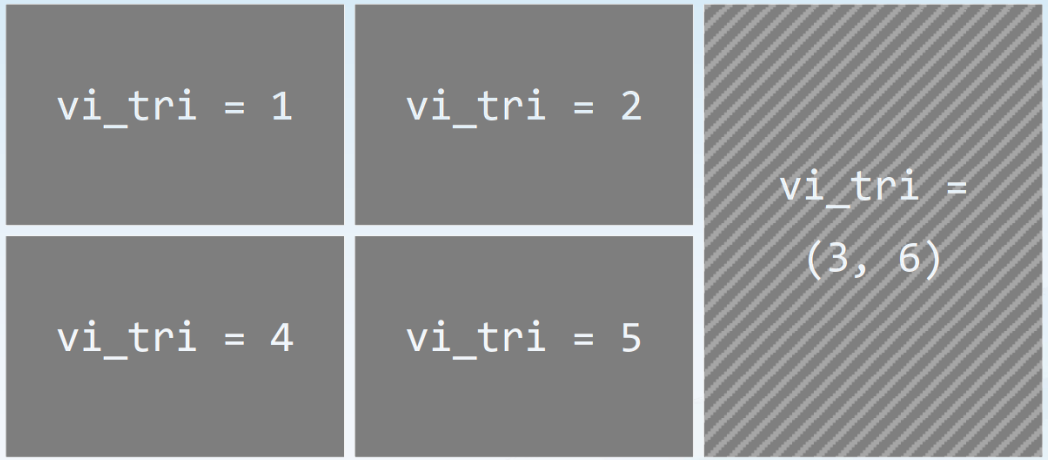



```
subplot(2,2,1)
subplot(221)
```

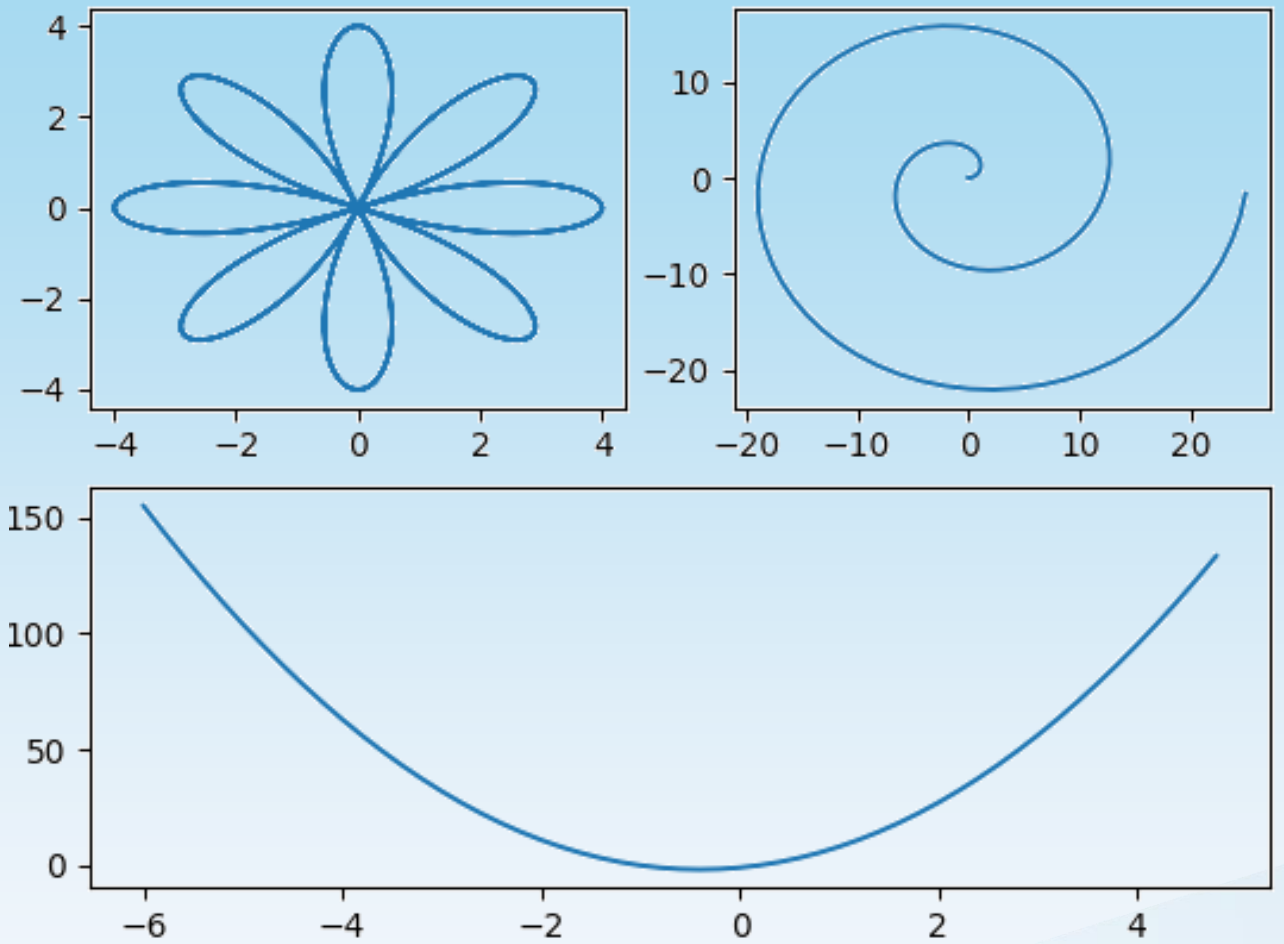
```
subplot(2,3,5)
subplot(235)
```




```
subplot(2,3,(3, 6))
```

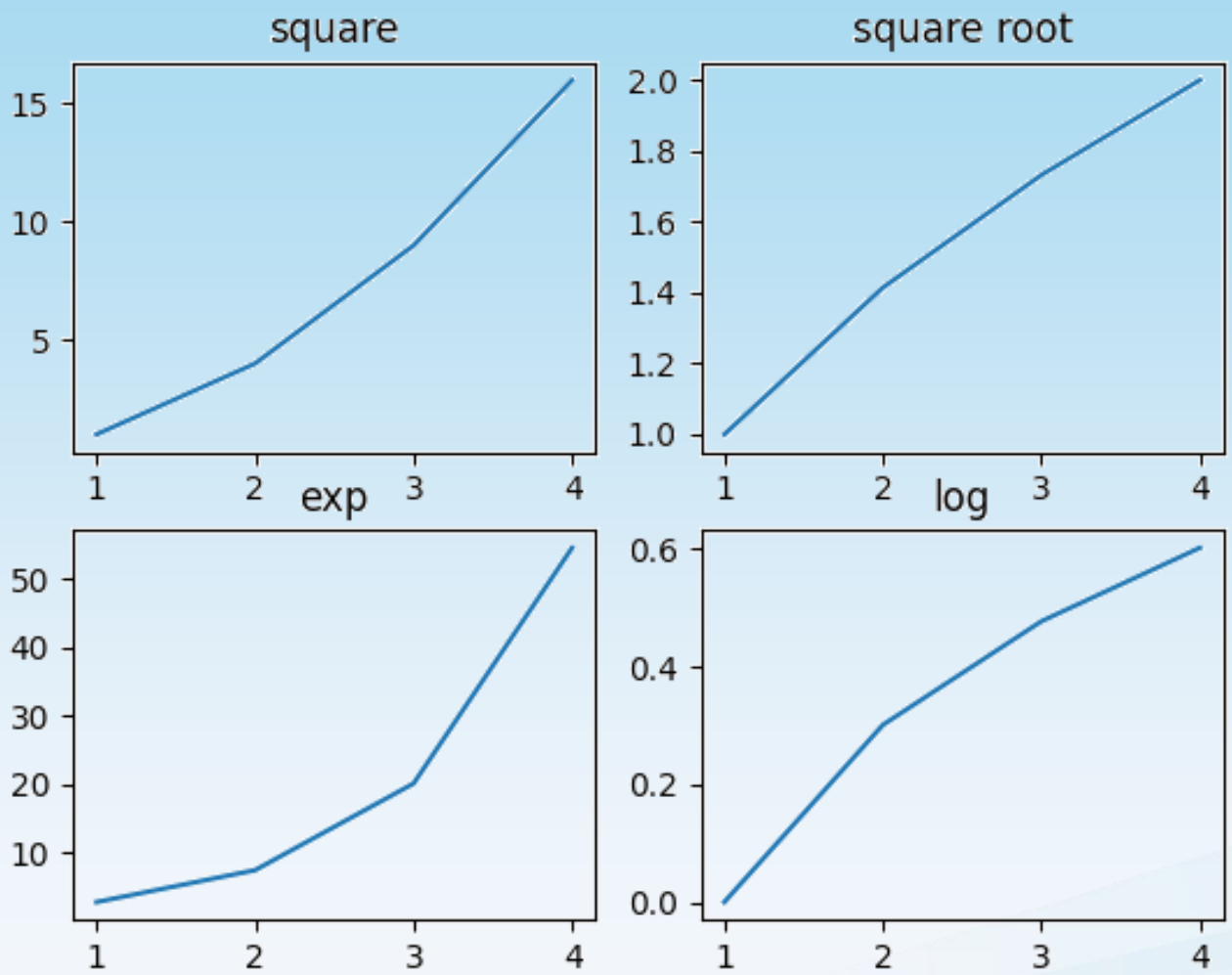


```
plt.subplot(2,2,1) # Plot a rose
a = 4; n = 4
theta = np.arange(0,4*np.pi,0.01)
r = a * np.cos(n*theta)
x = r*np.cos(theta); y = r*np.sin(theta)
plt.plot(x, y);
plt.subplot(2,2,2) # Plot a spiral
a = 2; theta = np.arange(0, 4*np.pi, 0.1)
r = a*theta
x = r*np.cos(theta); y = r*np.sin(theta)
plt.plot(x,y);
plt.subplot(2,2,(3,4)) # Plot a parabola
x = np.arange(-6, 5, 0.2); y = 5*x**2 + 4*x - 1
plt.plot(x, y);
plt.show()
```



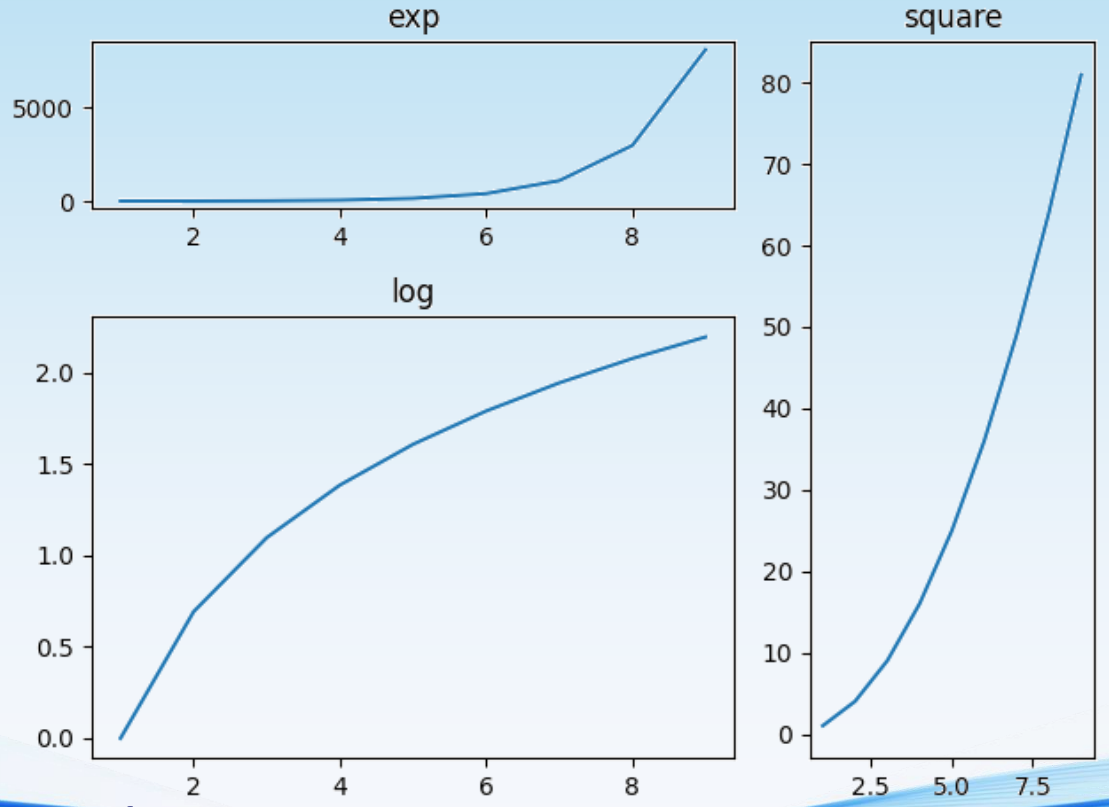
- Matplotlib cũng cũng cấp hàm subplots() giúp bố cục các subplot bao gồm cả figure bao quanh

```
subplots(nrows, ncols, index)
fig, a = plt.subplots(2,2)
x = np.arange(1,5)
a[0][0].plot(x,x*x)
a[0][0].set_title('square')
a[0][1].plot(x,np.sqrt(x))
a[0][1].set_title('square root')
a[1][0].plot(x,np.exp(x))
a[1][0].set_title('exp')
a[1][1].plot(x,np.log10(x))
a[1][1].set_title('log')
plt.show()
```

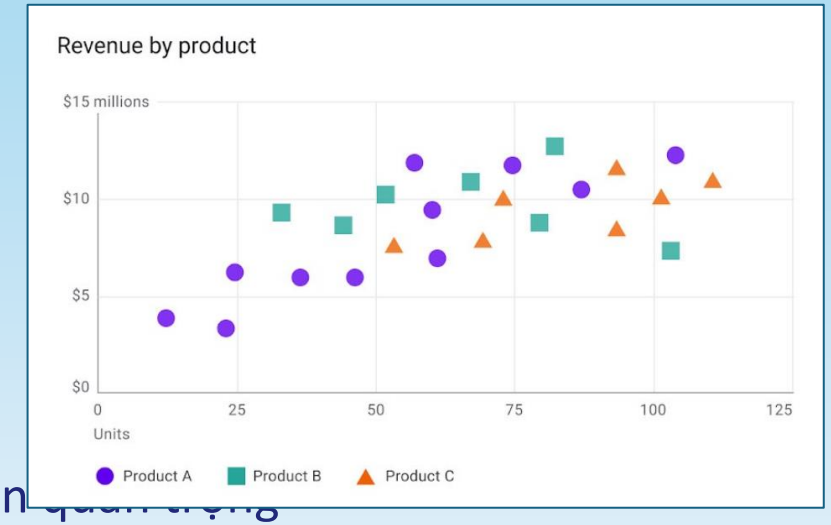


- subplot2grid() mang lại sự linh hoạt hơn cho các biểu đồ con

```
subplot2grid(shape, location, rowspan, colspan)
a1 = plt.subplot2grid((3,3),(0,0),colspan = 2)
a2 = plt.subplot2grid((3,3),(0,2), rowspan = 3)
a3 = plt.subplot2grid((3,3),(1,0),rowspan = 2, colspan = 2)
x = np.arange(1,10)
a2.plot(x, x*x)
a2.set_title('square')
a1.plot(x, np.exp(x))
a1.set_title('exp')
a3.plot(x, np.log(x))
a3.set_title('log')
plt.tight_layout()
plt.show()
```



- Các sử dụng màu sắc
  - ✓Chọn màu sắc phù hợp, không nên để quá nhiều màu sắc trên cùng một biểu đồ
  - ✓Phối màu hài hòa
  - ✓Sử dụng các màu tương phản
  - ✓Loại bỏ những màu không cần thiết, tránh để màu chồng chéo lên nhau
  - ✓Sử dụng màu sắc đúng mục đích, tránh lạm dụng gây tác dụng ngược
  - ✓Nên sử dụng các biểu tượng trên biểu đồ
  - ✓Độ đậm nhạt trên đường nét được sử dụng để lmaf nổi bật các thông tin quan trọng



- Để lựa chọn biểu đồ phù hợp cần xem xét các yếu tố:
  - ✓Loại biểu đồ: xem xét dữ liệu cần thể hiện để chọn biểu đồ thích hợp
  - ✓Tỷ lệ: xem xét giá trị và phân bố dữ liệu
  - ✓Chú ý đến đặc trưng chung và thế mạnh của từng loại biểu đồ

1. Thư viện Matplotlib trong Python cung cấp?

A. Thao tác dữ liệu B. Trực quan dữ liệu C. Học máy D. Xử lý văn bản

2. Phương thức để lưu biểu đồ thành tập tin?

A. savefig() B. saveplot() C. exportfig() D. downloadfig()

3. Phương thức subplot() dùng để?

A. Tạo nhiều biểu đồ con trên một biểu đồ B. Đặt tên cho biểu đồ con

C. Thêm biểu đồ con vào biểu đồ D. Chỉnh khoảng cách giữa các biểu đồ con

4. Phương thức imshow() dùng để?

A. Hiển thị ảnh B. Đặt tên cho ảnh C. Tạo scatter plot D. Biểu đồ histogram

5. Để khai báo thư viện Matplotlib ta sử dụng?

A. import matplot as plt B. import matplotlib.pyplot as plt

C. from matplotlib import pyplot as plt D. import matplotlib pyplot as plt

